Technical Architecture Document: TAD v1

Project: MidnightZK Off-Ramp SDK: ADA to Web2 Payments for Cash App, Wise, and Revolut
Version: 1.0
Date: April 2026
Prepared by: Nucast Labs Development Team

License: The SDK and this document are intended for the same open-source corpus as the repository. The MIT License text ships in the `LICENSE` file at the repository root.

---

1. Introduction

1.1 Purpose

This Technical Architecture Document, TAD v1, refines the Software Requirements Specification, SRS, into an implementable architecture: trust boundaries, component responsibilities, data flows, payment-rail integration including **sandbox** and **test** environments, and Cardano escrow plus Midnight ZK verification workflows.

Direction: Off-ramp only: user-side ADA moves through escrow toward a **fiat payout** on Web2 rails. **On-ramp** is fiat into ADA and is **not** in scope for this document or SDK surface described here.

1.2 Scope

In scope: non-custodial ADA escrow on Cardano, Midnight-based zero-knowledge proofs for payee and amount predicates, an off-chain SDK and orchestration layer, settlement confirmation against Web2 rails, and modular adapters for Cash App with Afterpay ecosystem APIs where applicable, Wise, and Revolut.

Out of scope, unchanged from PID and SRS except where clarified below: custodial **ADA** keys, **third-party open-market fiat liquidity** as a standalone product or exchange-style order book, licensing as a financial institution in the abstract, centralized exchange integrations.

Clarification: operator working capital: A supported deployment is **operator-funded fiat settlement**; the operator’s business rail or bank account sends funds to the **beneficiary**, then reconciles that outflow against escrow release of user ADA. That uses **operator float**, not user bank credentials on the rail. It is **not** the same as brokering unrelated third-party liquidity; legal and licensing posture for that leg remains a deployment responsibility.

1.3 Integration knowledge areas

Area	What integrators rely on
Cash App / Afterpay API, **sandbox and test**	Sandbox hosts, test customer flows, CVV-driven approve or decline behavior, and separation from production credentials.
Revolut, **sandbox setup**	Distinct sandbox credentials and API behavior versus production merchant configuration.
Wise, **API and sandbox**	Sandbox-funded test balances, personal or business API token issuance, and go-live checklists.
Midnight, **ZK, Compact, nodes**	Compact contract language, compiler-emitted circuits, zk-SNARK proving and verification, and proof-server style operations.

---

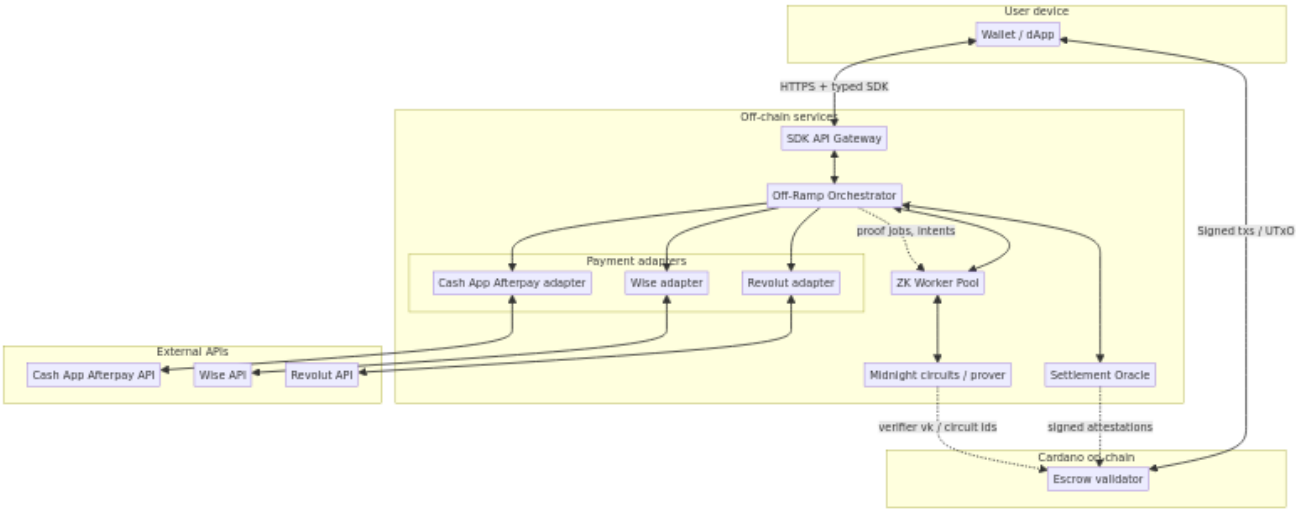
2. Architectural principles

- **Non-custodial:** Users sign Cardano transactions locally; the SDK never holds spend authority for ADA or fiat.
- **Privacy by design:** On-chain payloads favor commitments and hashes; proofs attest predicates without revealing raw PII.
- **Modular rails:** Cash App, Wise, and Revolut are **adapters** behind a stable internal port/interface.
- **Deterministic verification:** Escrow transitions and ZK verification are replay-safe and auditable.
- **Environment separation:** Sandbox versus production credentials, base URLs, and webhooks are isolated per deployment. Details appear in the API integration and smart contract sections below.

---

3. High-level architecture: on-chain and off-chain

The diagram below shows major components and trust boundaries. Settlement and proof generation run off-chain; escrow and proof verification policies anchor on Cardano; Midnight circuits and verifier artifacts bind the two.



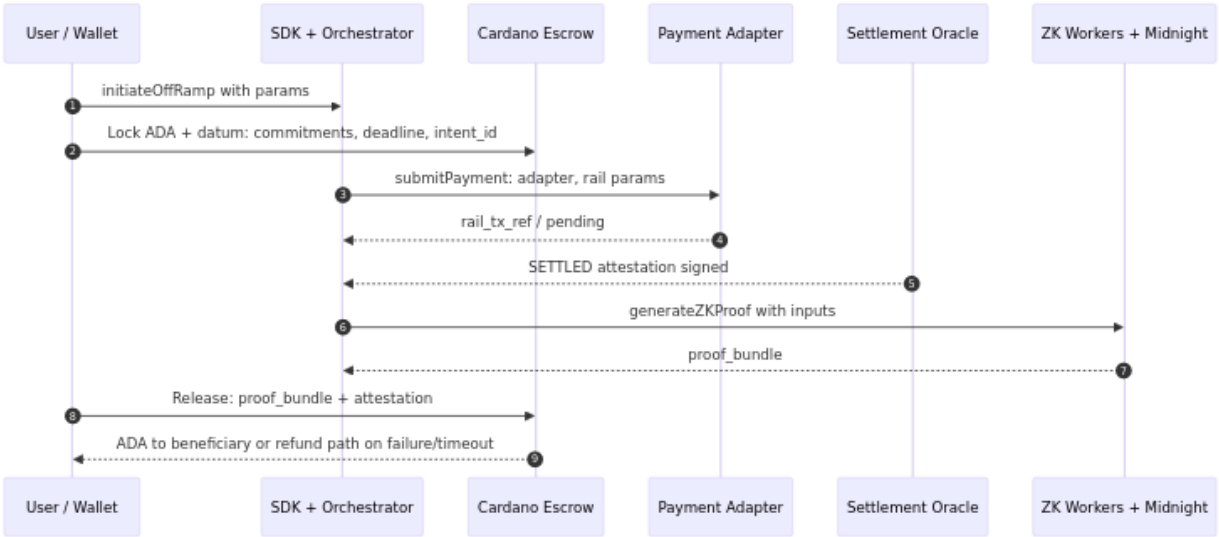
Component summary

Component	Responsibility
Wallet / dApp	Keys, signing, UX; optional wallet-level KYC; aligns with SRS.
SDK API Gateway	Stateless API, validation, idempotency, rate limits, routing.
Off-Ramp Orchestrator	End-to-end off-ramp state machine; coordinates escrow steps, adapters, proofs.
ZK Worker Pool	Proof generation workloads, circuit versioning, timeouts; SRS NFR targets proof generation under 50 seconds.
Midnight subsystem	Circuits for payee validity, amount correctness, optional compliance predicates.
Settlement Oracle	Normalizes rail-specific success or failure into signed canonical events bound to `intent_id`.
Adapters	Versioned HTTP clients, auth, retries, webhook verification, error mapping.
Escrow validator	Lock / release / refund; verifies commitments, deadlines, ZK bundle and oracle policy.

---

4. Data flow: ADA to USD

Steps for a fiat payout such as USD while ADA is locked in escrow. Optional pre-steps such as DEX or stablecoin flows are out of the critical escrow path unless added as extensions.



Off-ramp steps in prose

1. User obtains quote / plan from SDK; builds **LOCK** transaction: ADA to escrow UTxO with datum holding hashed payee, amount commitments, `adapter_id`, `deadline`, and `intent_id`.
2. Orchestrator stores ephemeral encrypted metadata with AES-256 at rest per SRS.
3. Adapter initiates fiat payout; receives provider reference as `rail_tx_ref`.

4. Oracle ingests webhooks or polling, verifies signatures, emits **SETTLED** or **FAILED** bound to `intent_id`.
5. ZK worker produces proof binding witness to public inputs.
6. **RELEASE** redeemer supplies proof and oracle chain; validator checks ZK, policy, and time; otherwise **REFUND** after timeout or failure.

4.1 Sender, beneficiary, operator: operator-funded fiat and who sees what

The following matches the **operator-funded** fiat leg: the **beneficiary** receives money from the **operator's** rail or bank identity, not from the end-user's personal bank sender profile on that rail. ADA remains user-controlled until escrow rules release it.

Role	Typical access	Notes
Sender, Cardano user / wallet	Own keys; full view of their ADA lock, refund, and release; any payee details they enter for the payout; status from SDK and wallet UI.	Does **not** need to share personal **bank** login with the rail if the operator originates fiat; they still trust the **operator** for correct payout and timing.
Beneficiary, fiat receiver	Sees **operator** as sender on bank or Wise-style rails; amount; reference text the operator or rail surfaces; may see limited metadata only.	Does **not** see the Cardano user's real-world identity **through the rail sender line** when the sender of record is the operator. They do **not** need a Cardano wallet.
Operator, runs gateway, orchestrator, adapters, oracle, optional proof servers	Maps `intent_id` to user wallet or account id; payout destination details needed to hit the rail; rail webhooks and `rail_tx_ref`; reconciliation between **outbound fiat** and **ADA escrow** release; AML and fraud tooling as required.	This party is the **only** one that joins **crypto identity** to **fiat destination** for a given intent in normal operation.

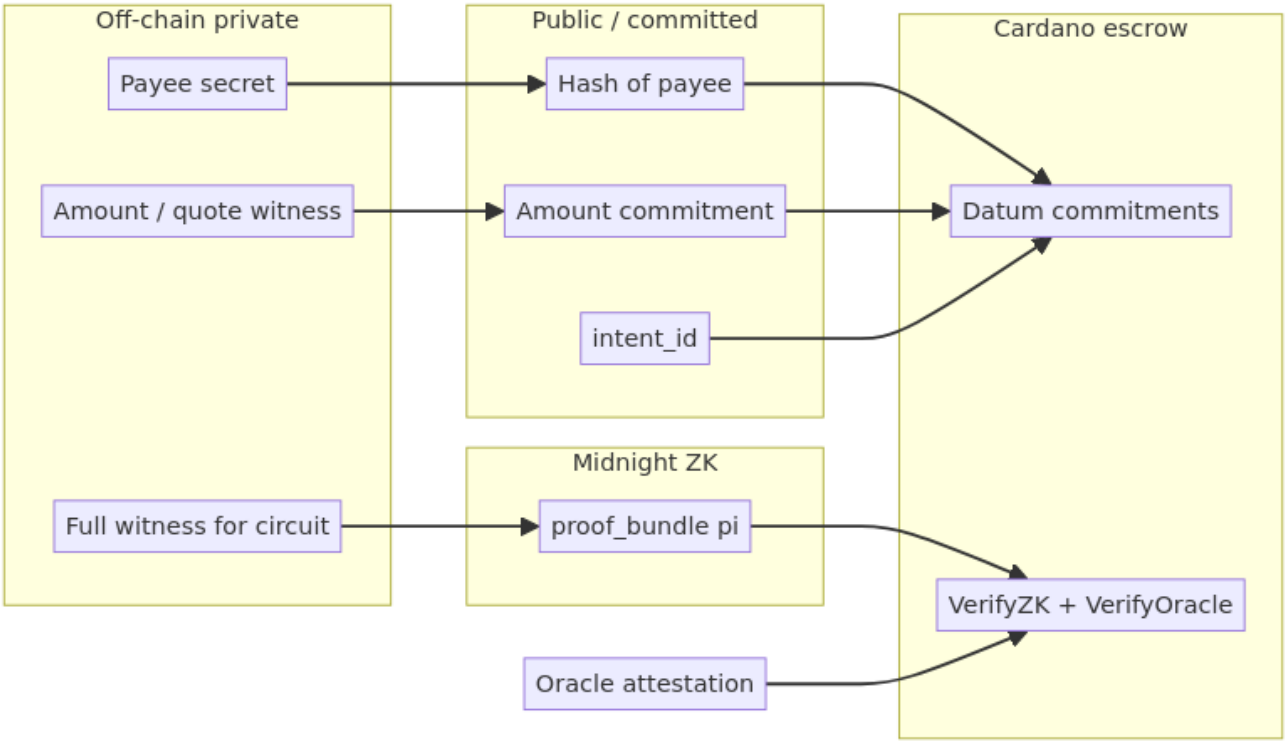
How privacy is implemented

- **Cardano on-chain:** commitments, hashes, `intent_id`, time bounds, escrow phase, opaque proof `pi`, pinned verifier material, and **signed oracle attestations**; not cleartext IBAN, cashtag, or legal name in datum. Observers and indexers see that **public** face only.
- **Midnight ZK:** prover shows witness satisfies predicates for payee binding, amount binding, and optional compliance flags, **without** putting the witness on Cardano.
- **Off-chain:** rail secrets, raw webhooks, and detailed logs live under operator and adapter policy; default posture is **minimize retention**, **hash** where durable logs are needed, and **AES-256** for ephemeral blobs.
- **Beneficiary-facing bank privacy:** achieved structurally when the **operator is the sender of record** on the fiat leg; it is **not** the same guarantee as hiding the payout from the operator.

---

5. Data flow: ZK-verified off-ramp

Separation of **private witness** versus **public inputs** aligns with Midnight's selective-disclosure approach.



5.1 Disclosure map: public versus ZK-protected private

The diagram below separates what Cardano observers can read, what stays inside the prover and wallet enclave, and how the zk-SNARK bridges them without moving cleartext PII into the datum.

- **Private** values never appear as readable strings in escrow datum; they enter only as commitments or digests the circuit relates to the proof.
- **Public** values are sufficient for any party to verify state transitions, time bounds, and that a valid proof exists for the pinned circuit.
- **Oracle material** is public to the escrow policy in the sense of signed attestations over `intent_id` and status, while raw rail payloads stay in the oracle and adapter tier off-chain, not in Cardano on-chain data.

Contract-side checks

- `VerifyMidnightProof` with `vk_hash`, `public_inputs`, and `pi` evaluates to true.
- `VerifyOracle` with `0`, `intent_id`, and `SETTLED` evaluates to true.
- Deadline and nonce / anti-replay rules satisfied

- No plaintext PII required on-chain; aligns with SRS privacy parameter table.

---

6. Midnight Compact contracts and zk-SNARK circuits

This section ties the off-ramp proof layer to Midnight's design: **Compact** contracts, circuits produced by the **Compact compiler**, and **zk-SNARK** proofs for succinct verification.

6.1 Compact contracts

Midnight provides **Compact**, a language for writing smart contracts on Midnight. The **Compact compiler** emits **zero-knowledge circuits** that prove ledger interactions satisfy contract rules without revealing the private witness. Tooling, security practices, and the standard library ship with the Compact toolchain.

Midnight describes contracts as combining **ledger** steps with **local** computation that supplies secrets. The constraints that link them feed a zk-SNARK. On chain, **entry points** are associated with **verification keys** so nodes check proofs instead of re-running sensitive local logic in the clear.

6.2 zk-SNARK circuits and the proving lifecycle

Midnight uses **zk-SNARKs:** succinct, non-interactive zero-knowledge arguments, so proofs stay small and verification stays efficient. A circuit encodes a statement as arithmetic constraints; a **prover** shows knowledge of a **private witness** and **public inputs** that satisfy those constraints; a **verifier** checks the proof against the **verification key** and public inputs only. The usual lifecycle is setup, circuit construction, witness and proof generation, then verification.

For this SDK, a "circuit" is any pinned constraint system the release identifies by **hash** and **version ID**, shipped as the compiler output from Compact sources in-repo. Proof workers run a prover compatible with that artifact; Cardano escrow policy or a linked verifier consumes the matching verification key material referenced on-chain as `vk_hash` or equivalent `circuit_id` fields in configuration.

6.3 Circuits proposed for the off-ramp

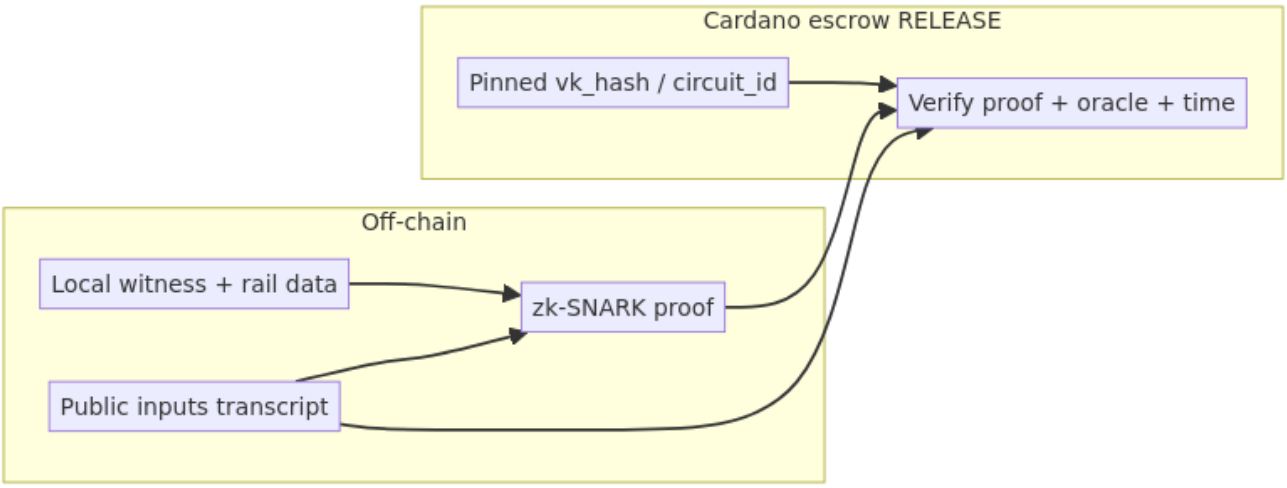
Circuit roles align with SRS payee, amount, and compliance predicates. Each release pins Compact modules, entry point names, and transcript fields in the circuit package.

Circuit purpose	Public inputs	Private witness
Payee binding	`intent_id`, `payee_commitment`, `adapter_id`, optional domain tag	cleartext payee handle or rail token, salt, path material used to form the commitment
Amount / quote binding	`amount_commitment`, rail quote digest, `intent_id`	principal, fees, rail references before hashing
Optional compliance predicate	constrained public flag or enum allowed by policy	jurisdiction-scoped attributes without raw PII
Transcript glue	sibling hashes or ledger anchors as designed	intermediate values only needed to satisfy constraints

The orchestration layer should pass a **settlement digest** derived from the oracle pipeline so constraints can bind Cardano escrow commitments to Web2 settlement events without placing bank-grade cleartext on Cardano on-chain.

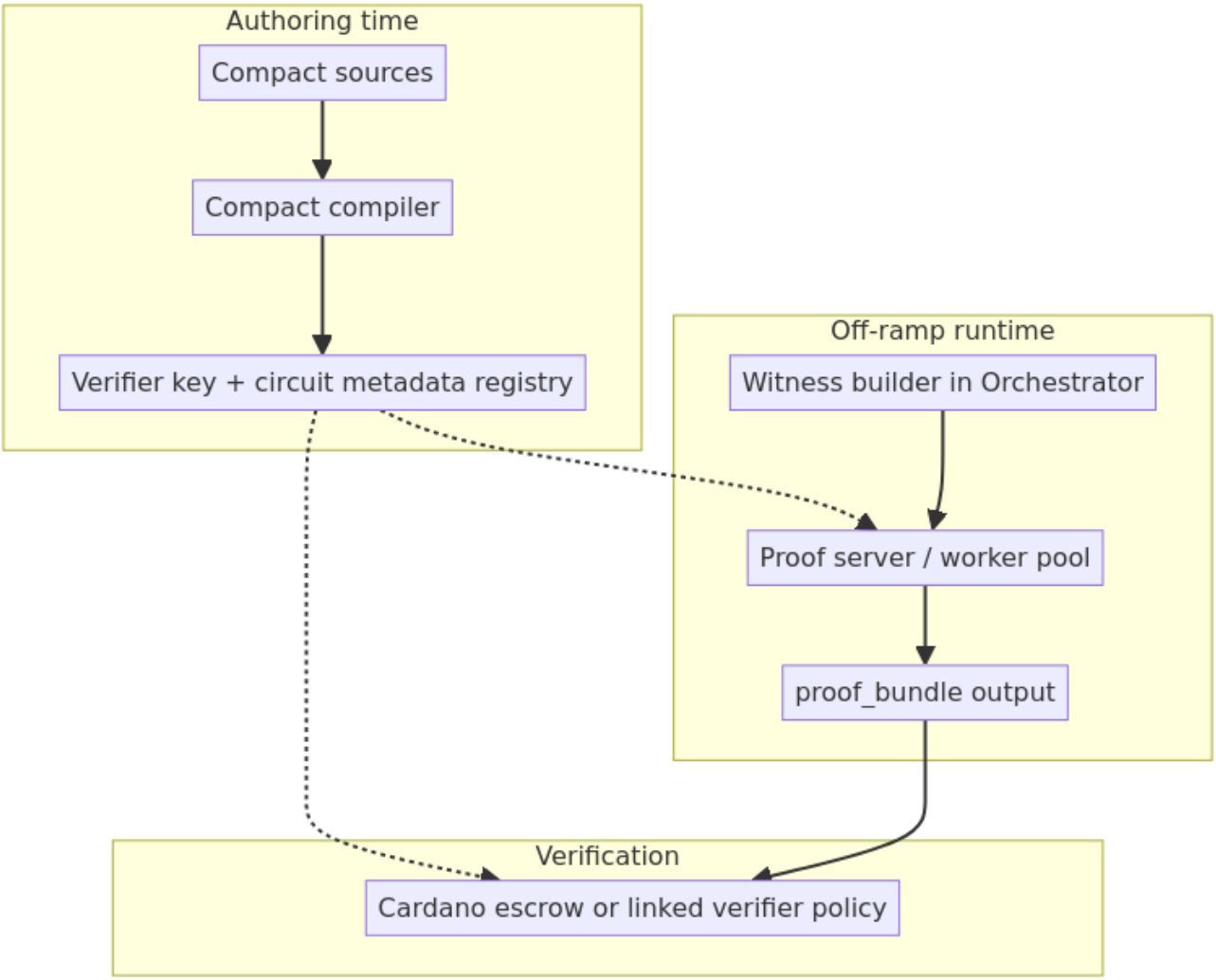
6.4 Public transcript plus proof: Midnight pattern mapped to Cardano

Midnight treats a contract call as a **public transcript** plus a **zero-knowledge proof** that the transcript respects the circuit for a chosen entry point; verification keys stand in for shipping full prover logic on chain. This SDK mirrors that split on the **RELEASE** path: redeemer arguments carry **public inputs** and a **proof** checked against a pinned `vk_hash`, while witness material never leaves the proof cluster and wallet enclave.



6.5 Proof operations and infrastructure

Treat proof generation like any heavyweight crypto job: isolated workers, pinned compiler and circuit artifacts, prove and verify timeouts, and metrics aligned with SRS performance targets. Operators may deploy **proof server** style services consistent with Midnight's operational layout. Wallet-facing APIs remain under **section 7**, `generateZKProof` and related methods.



---

7. API integration design

7.1 Internal SDK API for wallets

Method	Role
`initiateOffRamp`	Parameters: `params`. Create intent, return escrow plan and quote estimate.
`generateZKProof`	Parameters: `inputs`. Request proof job; return `proof_bundle`, commitments, `circuit_id`, `vk_hash`.
`submitPayment`	Parameters: `adapter`, `proof_bundle`. Drive rail-specific payout; return `rail_tx_ref`, status.
`confirmSettlement`	Parameters: `txHash`. Map chain / oracle state to settled flag and attestation id.

All requests should support **idempotency keys** and **correlation IDs** for horizontal scaling and safe retries.

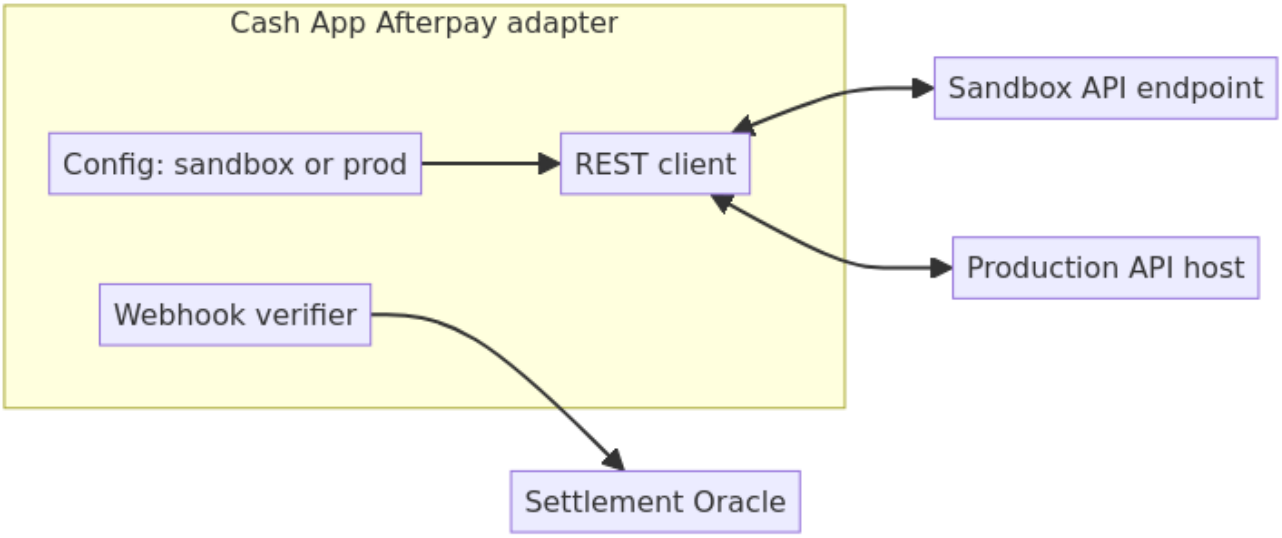
7.2 Cash App Afterpay: sandbox and integration notes

Official sandbox guidance, summarized without embedding live host strings:

- Sandbox API and Business Hub hosts differ from production; no real money moves in sandbox.
- Sandbox credentials and Business Hub access are coordinated through your **Cash App Afterpay account / delivery manager**.
- Test customers: unique email and phone; SMS is not sent in sandbox, use the vendor-designated static verification code where applicable.
- Test cards: specific CVV values trigger approve or decline paths on dummy card numbers per vendor rules.

Adapter design implications

- Separate **config profiles** for `sandbox` versus `production`: base URL, credentials, webhook secrets.
- Webhook signature verification before translating to oracle events.
- Store only **hashed** cashtag / handle commitments in durable logs; never log raw webhook PII in default pipelines.



7.3 Wise: sandbox and API access

Wise sandbox and API access, summarized:

- **Sandbox:** free sandbox account with email and password; no verification required for initial sandbox play.
- Sandbox environments are **pre-funded** with large test credit balances for integration work.
- **API tokens:** created from account **Settings → API tokens**. Two-step verification is required for personal tokens on real accounts; follow the Wise UI for sandbox versus live.
- Use the vendor **going live** checklist when moving from sandbox to production.

Adapter design implications

- Token scoped per environment; rotate via operator KMS.

- Recipient / quote / transfer endpoints abstracted behind `WisePort`; map errors to typed SDK errors.
- Commitments: hash of IBAN concatenated with salt, or equivalent account reference hash for on-chain binding; plaintext only in volatile memory.

7.4 Revolut: sandbox

Revolut provides a dedicated **sandbox** path for building and testing against their APIs before production.

Adapter design implications

- Separate API keys / OAuth config for sandbox versus merchant production.
- Regional availability and counterparty rules surfaced as typed errors to wallets.
- Store **hashed** user references; verify webhooks and signatures using keys and algorithms supplied during merchant integration at the oracle edge.

7.5 Settlement Oracle: operator-facing

- Ingress: rail webhooks + polling fallback.
- Egress: signed attestations consumed by the escrow **RELEASE** path, e.g. Ed25519 over canonical payload fields `intent_id`, status, `rail_tx_ref`, expiry, chain id.
- **Replay protection:** nonce store keyed by `intent_id` together with `rail_event_id`.

---

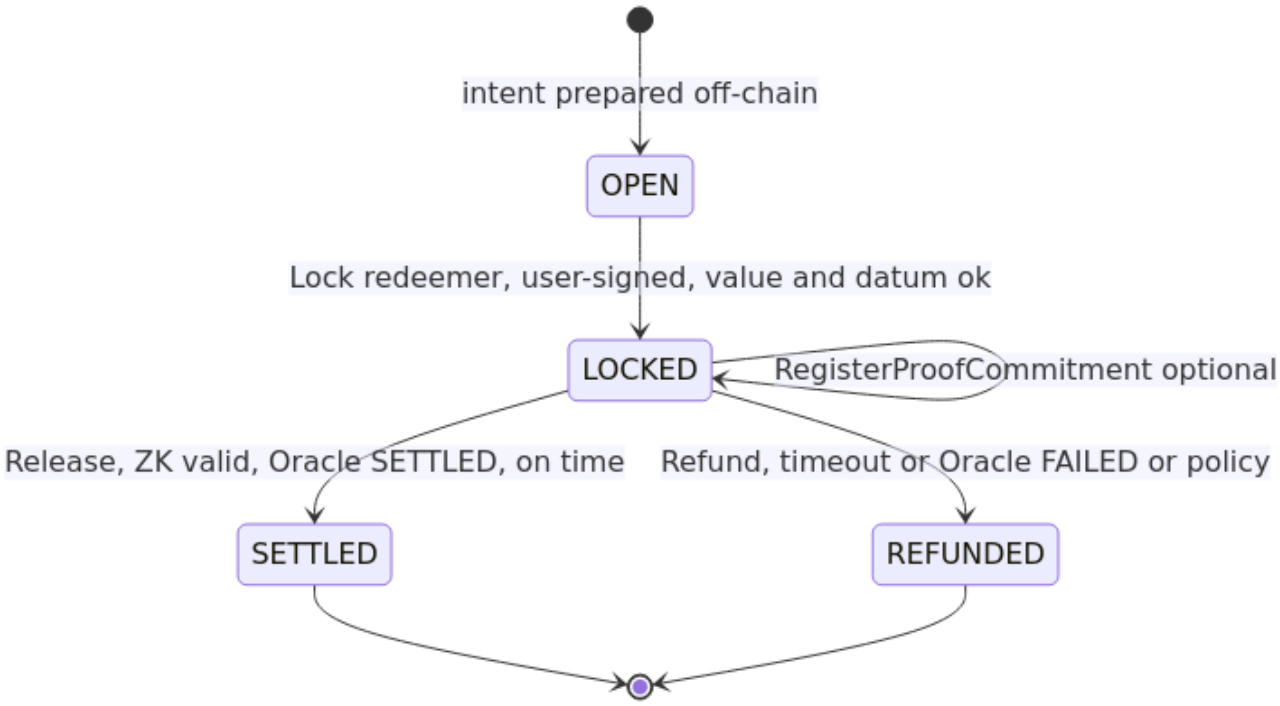
8. Smart contract interactions

8.1 Escrow datum

Escrow datum fields:

```
EscrowDatum {
  intent_id            : ByteString
  locker               : PubKeyHash
  beneficiary          : PubKeyHash
  ada_locked           : Integer
  payee_commitment     : ByteString
  amount_commitment    : ByteString
  adapter_id           : Enum   cashapp | wise | revolut
  created_at           : POSIXTime
  deadline             : POSIXTime
  proof_commitment     : ByteString   optional single-use binding
  nonce                : ByteString
}
```

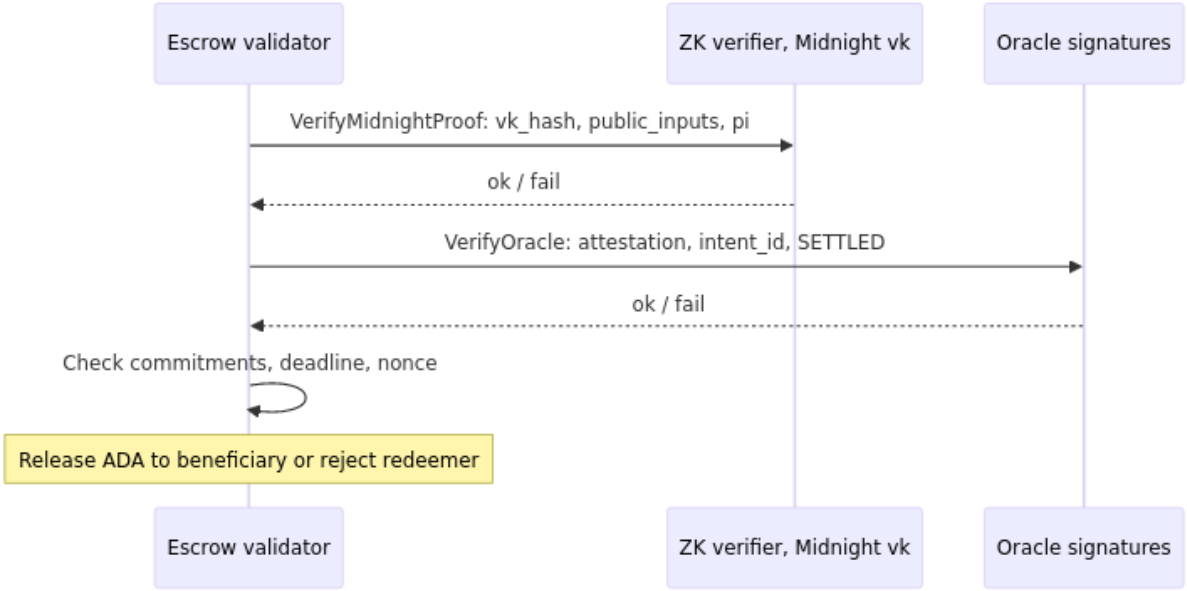
8.2 State machine



Invariants

- No `RELEASE` without successful ZK verification and valid oracle attestation.
- No `REFUND` after `SETTLED`.
- Time and nonce monotonicity enforced in validator.

8.3 ZK payee verification: on-chain steps



---

9. Scalability

- Stateless API tier + horizontal autoscaling; no sticky sessions required.
- Proof queue with worker autoscaling; circuit version pinned per job type.

- Per-adapter rate limits using token bucket plus global budget; circuit breakers on rail outages.
- Sharded persistence by `intent_id` prefix; short-TTL cache for read-heavy status with strong consistency on settlement writes.

---

10. Privacy

10.1 Cardano public, off-chain confidential, Midnight ZK private

Use this split when designing logs, dashboards, and incident response: **Cardano on-chain** shows only what observers must verify; **off-chain** systems hold rail secrets and reconciliation detail; **Midnight ZK** sees the full private witness and emits only the proof and public inputs bound to Cardano.

- On-chain: commitments and state transitions only; aligns with SRS privacy parameter table.
- Off-chain: AES-256 at rest for ephemeral blobs; strict TTL; KMS rotation.

- Logs: structured redaction; default **no** raw webhook body retention.
- Optional wallet-level KYC remains outside core protocol verification.

---

11. Security

Threat	Mitigation at architecture level
Forged oracle	Multi-sig / threshold signing in production; key in HSM/KMS; clock skew bounds.
Replayed webhooks / attestations	HMAC + nonce store + event id deduplication.
Malformed / wrong-circuit proofs	Pinned `vk_hash` / `circuit_id`; reject unknown versions.
MITM to rails	TLS 1.2+; optional mTLS internal; certificate pinning policy for adapters, operator choice.
Supply chain	Pinned deps, MIT-compatible license audit in CI, reproducible builds, recommended.

---

12. Open source and MIT alignment

- Repository license: MIT; text in `LICENSE` at repository root.
- Publish **OpenAPI** specs with the SDK; SPDX headers in source; **NOTICE** for third-party attribution where required.
- Public GitHub, tagged releases, and issue-tracked architectural changes.

---

13. Document control

Version	Date	Notes
1.0	April 2026	Initial TAD; Mermaid diagrams; vendor integration summaries; Midnight Compact and zk-SNARK section.